

Layout delle immagini nei post

Filosofia: no-class HTML

In questo progetto il CSS non usa classi (vedi branch `noCSS-experiment`). Le variazioni di layout delle immagini si esprimono tramite **attributi HTML semantici**, non classi. Il meccanismo scelto è `data-layout` su `<figure>` o `<img>`.

Vantaggi rispetto alle classi:

- Il Markdown/HTML resta leggibile e auto-documentante
- Il CSS seleziona per struttura e attributo (`figure[data-layout="wide"]`), non per nome arbitrario
- Si allinea alla stessa logica del resto del sistema (es. `nav a[lang]` per il language switcher)

Varianti disponibili

Un transform in `.eleventy.js` converte automaticamente la sintassi Markdown abbreviata in HTML con attributi `data-layout`. In alternativa si può scrivere HTML grezzo direttamente nel `.md`.

Sintassi Markdown	HTML generato
<code>![descrizione](img.jpg)</code>	<code></code>
<code>![masthead\ alt](img.jpg)</code>	<code><figure data-layout="masthead"></figure></code>
<code>![cover\ alt](img.jpg)</code>	<code><figure data-layout="cover"></figure></code>
<code>![wide\ alt](img.jpg)</code>	<code><figure data-layout="wide"></figure></code>
<code>![half\ alt](img.jpg)</code>	<code><figure data-layout="half"></figure></code>
<code>![inline\ alt](img.jpg)</code>	<code></code>
<code>![inline-right\ alt](img.jpg)</code>	<code></code>

La parte prima del `|` è il layout, la parte dopo è il testo alternativo (`alt`).

I layout `cover`, `wide`, `half` rimuovono automaticamente il `<p>` wrapper che Markdown aggiunge.

`masthead` — hero a inizio articolo

Immagine full-bleed ad altezza fissa (55vh) che occupa tutta la larghezza sopra il titolo.
L'altezza è calibrata perché il titolo **h1** rimanga visibile al primo scroll sia su desktop che mobile.
Va messa come **primissima riga del contenuto**, prima del kicker e del titolo.

```
![masthead|Descrizione immagine hero](/percorso/immagine.jpg)

::kicker opzionale::
# Titolo dell'articolo
```

Focal point — controllare il crop su mobile

Con **object-fit: cover** e altezza fissa, l'immagine viene ritagliata automaticamente.
Il punto di fuoco predefinito è il centro (**center center**).
Se il soggetto principale è fuori centro, si può spostare il focal point con la CSS custom property **--focal** direttamente sull'attributo **style** della **<figure>**.

La sintassi richiede HTML grezzo nel **.md** (non funziona con **![masthead|...]**):

```
<figure data-layout="masthead" style="--focal: 30% 60%">
  
</figure>
```

I valori di **--focal** sono coordinate **x% y%** come in **object-position**:

Valore	Effetto
center center	default — mostra il centro
left center	ancora l'immagine a sinistra
right center	ancora l'immagine a destra
50% 20%	ancora verso l'alto (utile per paesaggi con soggetto in alto)
30% 70%	ancora in basso a sinistra

Questo approccio risolve la maggior parte dei casi senza dover preparare due immagini separate.

Art direction vera (due crop reali)

Per mobile e desktop con composizioni completamente diverse (es. ritratto su mobile, panoramica su desktop) si possono usare due immagini sorgente con HTML diretto:

```
<figure data-layout="masthead">
  <picture>
    <source media="(max-width: 480px)" srcset="/percorso/immagine-
mobile.jpg">
```

```

</picture>
</figure>
```

Richiede di preparare manualmente le due versioni croppate.

Nessun attributo — immagine standard

```
![Didascalia dell'immagine](percorso/immagine.jpg)
```

Comportamento: larghezza massima del container, ombra sottile, border-radius.

Con didascalia visibile (HTML diretto):

```
<figure>
  
  <figcaption>Testo didascalia</figcaption>
</figure>
```

cover — full bleed

Esce dal container e occupa tutta la larghezza del viewport.

```
![cover|Descrizione dell'immagine](percorso/immagine.jpg)
```

wide — più larga del testo

Leggermente più larga della colonna di testo (aggiunge ~6rem ai lati).

Su mobile ritorna alla larghezza normale.

```
![wide|Descrizione dell'immagine](percorso/immagine.jpg)
```

half — metà colonna, flotta a sinistra

Occupi il 50% della larghezza e lascia scorrere il testo a destra.

Su mobile torna a larghezza piena, senza float.

```
! [half|Descrizione dell'immagine] (percorso/immagine.jpg)
```

inline — piccola, dentro il testo, sinistra

Immagine che flotta a sinistra dentro un paragrafo. Il testo scorre a destra.

```
! [inline|Descrizione] (percorso/immagine.jpg) Qui continua il testo del paragrafo...
```

inline-right — piccola, dentro il testo, destra

Come **inline** ma flotta a destra. Il testo scorre a sinistra.

```
! [inline-right|Descrizione] (percorso/immagine.jpg) Qui continua il testo del paragrafo...
```

Ottimizzazione automatica (WebP + srcset)

Il transform `imgOptimize` in `.eleventy.js` processa automaticamente ogni ``:

- Converte in **WebP** (con fallback JPEG per browser vecchi)
- Genera **srcset** con più larghezze, calibrate per layout
- Aggiunge `loading="lazy"` su tutto tranne il masthead
- Aggiunge `decoding="async"` su tutti
- Wrappa in `<picture>` trasparente al layout (`display: contents`)
- Risultati **cachati su disco** in `.cache/` — la seconda build è veloce

Le larghezze generate per layout:

Layout	Widths (px)	Sizes hint
<code>masthead, cover</code>	600, 1200, 1440, 1920	<code>100vw</code>
<code>wide</code>	400, 800, 1100, 1440	<code>calc(100% + 6rem)</code>
<code>half</code>	200, 400, 500, 960	<code>50vw</code>
<code>inline, inline-right</code>	150, 300, 600	<code>200px</code> (30vw su mobile)
standard	400, 800, 960, 1440	<code>960px</code>

I widths includono sempre versioni 2x per schermi retina. Il browser sceglie automaticamente la dimensione più adatta in base a DPR e larghezza del viewport.

Il JPEG nei `<source>` è un fallback necessario — i browser moderni (Chrome, Firefox, Safari 14+, Edge) caricano sempre il WebP. Per verificare quale formato viene effettivamente caricato: DevTools → Network → filtra "Img" → colonna "Type".

I file ottimizzati vengono scritti in `_site/assets/img/` (non in `assets/`).

Dove sono le definizioni CSS

File: `assets/style.css`, sezione `/* — Image sizing system: figure[data-layout] */`.

```
figure[data-layout="cover"] { ... }
figure[data-layout="wide"]  { ... }
figure[data-layout="half"]  { ... }
img[data-layout="inline"]   { ... }
img[data-layout="inline-right"] { ... }
```

Domanda aperta: alternative no-class pure

L'attributo `data-layout` è una soluzione pragmatica ma è comunque un attributo inventato, non semantico HTML nativo. Alcune alternative da considerare:

Approccio	Esempio	Pro	Contro
<code>data-layout</code> (attuale)	<code><figure data-layout="wide"></code>	Esplicito, leggibile	Non nativo HTML
Posizione nel DOM	<code>figure:first-child</code>	Zero markup extra	Dipende dall'ordine, fragile
Elemento contenitore	<code><aside><figure></code>	Semantico	<code>aside</code> ha significato diverso
<code><figure></code> + <code><figcaption></code> con lunghezza	—	—	Non influenza il layout

Per ora `data-layout` è il compromesso migliore: è un attributo, non una classe, e si seleziona con `[attr]` selector, coerente con la filosofia del progetto.